

Weighted Batch Threshold Encryption

No Author Given

No Institute Given

Abstract. Batch threshold encryption allows a committee to decrypt a selected batch of ciphertexts using one short pre-decryption key per party, while ciphertexts outside the batch remain private. This makes batch threshold encryption attractive for building encrypted mempools for blockchains, where validators reveal transactions selected for a block while pending transactions remain private. However, in proof-of-stake systems such as Solana, authorization depends on each validator’s stake, which motivates a weighted version of batch threshold encryption. Naive approaches to weighting existing batch threshold encryption schemes lead to communication costs proportional to the weight assigned to each party. Existing weighted constructions of batch threshold encryption with weight-independent communication either require batch/epoch labels, or are not censorship-resistant.

We present the first weighted batch threshold encryption schemes that require neither batch/epoch labels nor encryption-time ciphertext indices, while keeping each party’s communication independent of its weight. We obtain two constructions by virtualizing prior schemes: the **BTX** construction of Agarwal et al. and the partial fraction construction of Boneh et al. In both schemes’ decryption procedures, each real party communicates a single group element for an entire batch, regardless of its assigned weight. We prove correctness, robustness, and security under weighted variants of the source assumptions, and analyze these assumptions in generic bilinear groups. For the more efficient **BTX**-based construction, we also give a specialized distributed key generation protocol that avoids a general purpose MPC-based setup, as the source schemes require. Finally, we implement our schemes in Rust to demonstrate their practical performance at parameter sizes relevant to encrypted mempools.

Keywords: Threshold encryption · Batch encryption · Bilinear groups

1 Introduction

2 Preliminaries

2.1 Basic preliminaries

Bilinear groups. Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, e) \leftarrow \text{GrpGen}(1^\lambda)$ be the descriptions of prime-order groups, their identity elements, and a bilinear pairing e . We use additive notation for group operations, so $e([a]_1, [b]_2) = [ab]_T$. All algorithms implicitly receive the group descriptions, and all scalar arithmetic is over $\mathbb{F}_p$.

Weights and sharing. Let N denote the number of real parties in our scheme, and W denote the total weight assigned to the N parties. Party j has positive integer weight w_j and owns a set $\Omega_j \subset \mathbb{F}_p^*$ of w_j evaluation points used for Shamir secret sharing. The sets $(\Omega_j)_{j \in [N]}$ are disjoint. For a set of parties T , set the notation

$$\Omega_T = \bigcup_{j \in T} \Omega_j, \quad \text{wt}(T) = |\Omega_T| = \sum_{j \in T} w_j, \quad W = \text{wt}([N]) = |\Omega_{[N]}|.$$

For a corruption bound $t < W$, a set T is authorized if $\text{wt}(T) > t$ and unauthorized if $\text{wt}(T) \leq t$. For authorized T , let $\lambda_{T,\omega}$ be the Lagrange coefficient at zero for each $\omega \in \Omega_T$ over interpolation set T .¹ For every polynomial F of degree at most t ,

$$F(0) = \sum_{\omega \in \Omega_T} \lambda_{T,\omega} F(\omega).$$

The party which owns a share $\omega \in \Omega_j$ is denoted by $\text{own}(\omega) = j$.

2.2 Weighted batch threshold encryption

We now define the syntax and security properties of our scheme. The following syntax extends the batch threshold encryption definition of Boneh et al. [9] to a weighted access structure.

Definition 1 (Weighted BTE syntax). A weighted batch threshold encryption scheme WBT with maximum batch size ℓ , total weight W , threshold t , and message space $\mathcal{M}$ consists of the following PPT algorithms:

- $\text{Setup}(1^\lambda, 1^\ell, 1^W, 1^t, (w_j)_{j \in [N]}) \rightarrow (\text{pp}, \text{ek}, \text{dk}, (\text{sk}_j, \text{vk}_j)_{j \in [N]})$: Given security parameter λ , maximum batch size ℓ , total weight W , threshold t and per-party weights w_j , outputs public parameters pp , a public encryption key ek , public decryption key dk , and per-party secret/verification keys $(\text{sk}_j, \text{vk}_j)_{j \in [N]}$. pp is an implicit input to the subsequent algorithms.
- $\text{Enc}(\text{ek}, m) \rightarrow \text{ct}$: encrypts m using ek .
- $\text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}) \rightarrow \text{sbk}_j$ or $\perp$: Using a party's secret key sk_j , and a set of up to ℓ ciphertexts, outputs the corresponding **short** pre-decryption per-party key sbk_j .
- $\text{Dec}(\text{dk}, T, (\text{sbk}_j)_{j \in T}, (\text{ct}_i)_{i \in [\ell]}) \rightarrow (m_i)_{i \in [\ell]}$ or $\perp$: decrypts a batch of ciphertexts $(\text{ct}_i)_{i \in [\ell]}$ using a collection of pre-decryption keys $(\text{sbk}_j)_{j \in T}$ corresponding to a set of parties T with total weight greater than t . Decryption first verifies each ciphertext and submitted share. It may discard invalid shares, but it can only return messages if the accepting set is authorized.

Definition 2 (Correctness). A weighted BTE scheme is correct if, for every authorized T , every message vector $(m_i)_{i \in [\ell]}$, honestly generated keys and ciphertexts, and honest shares $\text{sbk}_j \leftarrow \text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]})$ for $j \in T$, decryption returns $(m_i)_{i \in [\ell]}$, except with negligible probability.

¹ i.e. $\lambda_{T,\omega}$ is the Lagrange indicator function $\prod_{u \in \Omega_T, u \neq \omega} \frac{X-u}{\omega-u}$ evaluated at $X = 0$.

We adapt the static batch chosen-ciphertext game of Boneh et al. [9] to this context by replacing cardinality with weight. The formulation below gives the adversary all honest pre-decryption shares, which is at least as strong as giving it the final decryption output.

Definition 3 (Static WB-IND-CCA security). *The game is as follows:*

1. The challenger generates keys/parameters via **Setup** and gives (ek, dk) to $\mathcal{A}$. The adversary then fixes an (unauthorized) corruption set $\mathcal{C} \subseteq [N]$ with $\text{wt}(\mathcal{C}) \leq t$ and receives $(\text{sk}_j)_{j \in \mathcal{C}}$.
2. $\mathcal{A}$ may submit batches of at most ℓ ciphertexts. For each batch, the challenger returns $\text{PreDec}(\text{sk}_j, \cdot)$ for every $j \notin \mathcal{C}$.
3. The adversary submits $m_0, m_1 \in \mathcal{M}$ of equal length. The challenger samples $b \leftarrow \{0, 1\}$ and returns $\text{ct}^* \leftarrow \text{Enc}(\text{ek}, m_b)$.
4. $\mathcal{A}$ may submit batches of at most ℓ ciphertexts which do not contain ct^* . For each batch, the challenger returns $\text{PreDec}(\text{sk}_j, \cdot)$ for every $j \notin \mathcal{C}$.
5. $\mathcal{A}$ outputs bit b' .

The advantage of $\mathcal{A}$ against the weighted BTE scheme WBT is

$$\text{Adv}_{\text{WBT}, \mathcal{A}}^{\text{WB-IND-CCA}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|.$$

The scheme is secure if this advantage is negligible for every PPT adversary, polynomial ℓ, N, W .

[Aditi: shouldn't we allow the adversary to choose ℓ, N, W at the beginning of the game? otherwise, we should parametrize the game/advantage defn w.r.t. ℓ, N, W]

A weighted BTE scheme must be *efficient* in that (i) the size of the pre-decryption key shares $(\text{sbk}_j)_{j \in [N]}$ is succinct in the maximum batch size ℓ and weight assigned to party j , and (ii) the size of the ciphertexts is succinct in the number of parties. [Sasha: Do we also need an efficiency property in that the ciphertext size is also succinct in the number of shares?] [Nusret: Good catch! I added a sentence, please check.]

2.3 Robustness

A weighted batch threshold encryption scheme satisfies share robustness if it has accurate share verification and accurate blaming. We adapt the definitions of these properties from [10, Definitions 22.13 & 22.14].

[Aditi: why dont we explicitly define a **PreDecVerify** or **ShareVerify** algorithm to the syntax, and refer to it when talking about share verification...]

Definition 4 (Accurate blaming). *For a decryption call, let*

$$\text{Blame}(\text{dk}, T, (\text{sbk}_j)_{j \in T}, (\text{ct}_i)_{i \in [\ell]}) \subseteq T$$

denote the set of parties whose pre-decryption shares are rejected by the share verification step of Dec. A weighted BTE scheme ensures accurate blaming if, for every honestly generated parameter tuple, every ciphertext batch $(\text{ct}_i)_{i \in [\ell]}$, every authorized submitted set $T \subseteq [N]$, every collection of shares $(\text{sbk}_j)_{j \in T}$, and every $j \in T$, membership

$$j \in \text{Blame}(\text{dk}, T, (\text{sbk}_k)_{k \in T}, (\text{ct}_i)_{i \in [\ell]})$$

implies

$$\Pr [\text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}) = \text{sbk}_j] = 0,$$

where the probability is over the internal randomness of PreDec.

Definition 5 (Consistency). The game is as follows:

1. The challenger invokes $(\text{ek}, \text{sk}_1, \dots, \text{sk}_N, \text{dk}) \leftarrow \text{Setup}(1^\lambda, 1^\ell, 1^W, 1^t, (w_j)_{j \in [N]})$ and returns the output to Adv.
2. Adv sends $((\text{ct}_i)_{i \in [\ell]}, S_1, \{\text{sbk}_j^1\}_{j \in S_1}, S_2, \{\text{sbk}_j^2\}_{j \in S_2})$, where $(\text{ct}_i)_{i \in [\ell]}$ is a batch of ℓ ciphertexts, $S_1, S_2 \subseteq [N]$ are (authorized) sets of parties with weight at least t , and $\{\text{sbk}_j^1\}_{j \in S_1}$ and $\{\text{sbk}_j^2\}_{j \in S_2}$ are pre-decryption shares.
3. Define $\text{Dec}(\text{dk}, S_1, (\text{sbk}_j)_{j \in S_1}, (\text{ct}_i)_{i \in [\ell]}) = (m_i^1)_{i \in [\ell]}$ and $\text{Dec}(\text{dk}, S_2, (\text{sbk}_j)_{j \in S_2}, (\text{ct}_i)_{i \in [\ell]}) = (m_i^2)_{i \in [\ell]}$. Adv wins the game if $(m_i^1)_{i \in [\ell]} \neq (m_i^2)_{i \in [\ell]}$ and all pre-decryption shares are accepted by the share verification step of Dec.

The advantage of Adv, denoted by $\text{Adv}_{\text{WBT}, \mathcal{A}, \ell, N, W}^{\text{Cons}}$ is the probability that it wins the game. The WBT scheme is said to be consistent if this advantage is negligible for every PPT adversary, and polynomial ℓ, N, W .

2.4 Non-Interactive Zero Knowledge Proofs (NIZKs)

Both constructions use an online simulation-extractable NIZK Π_{DL} . For the group element $C \in \mathbb{G}_1 \times \mathbb{G}_T$, it proves the relation

$$R_{\text{DL}} = \{(C, r) : C = [r]_1\}.$$

It allows the extractor in our proofs to recover the scalar values needed to simulate a pre-decryption share for a given ciphertext. We require Π_{DL} to be complete and zero-knowledge as defined below. We also require simulation extractability: after seeing simulated proofs, every fresh accepting statement–proof pair has an extractable r satisfying $C = [r]_1$, except with negligible probability. In particular, a new proof for a previously simulated statement is extractable unless it is the simulated proof itself. [Sasha: I'm not good with simulation extractability definitions. Is this last sentence AI slop or a consideration people would care about?]

Definition 6 (Non-Interactive Zero Knowledge Proofs). Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be a relation over sets $\mathcal{X}$ and $\mathcal{W}$. A NIZK scheme NIZK for $\mathcal{R}$ ² is a tuple of PPT algorithms, $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ defined as follows.

² Later, we will instead have a family of indexed relations to capture public parameters with which the relation is specified.

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$: given the security parameter λ , outputs a common reference string crs . The algorithms below implicitly take crs as input.
- $\text{Prove}(x, w) \rightarrow \text{pf}$: given a statement $x \in \mathcal{X}$ and a witness $w \in \mathcal{W}$, outputs a proof pf .
- $\text{Verify}(x, \text{pf}) \rightarrow 0/1$: given a statement $x \in \mathcal{X}$ and a proof pf , outputs 0 or 1.

In this work, we require *simulation-extractability* (SE), which implies special soundness and knowledge soundness. Informally, a NIZK satisfies SE if a prover cannot generate new accepting proofs (Verify outputs 1) even with access to simulated proofs if it does not *know* a valid witness. In other words, we can construct an extractor that works even using simulated proofs. Formally, a NIZK is an SE-NIZK if it satisfies the following additional property (based on [?, §3.2]):

Definition 7 (Simulation-extractability). A NIZK scheme for $\mathcal{R}$, $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$, is said to be *simulation-extractable* or an *SE-NIZK* if there exists a PPT extractor Ext and a PPT simulator $\text{Sim} = (\text{SimSetup}, \text{SimProve})$ such that for all PPT adversaries Adv , there exists a negligible function $\text{negl}(\lambda)$ for which, the following holds:

$$\Pr \left[\begin{array}{l} \text{Verify}(x^*, \pi^*) = 1 \wedge \\ x^* \notin Q \wedge \\ (x^*, w^*) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} (\text{crs}, \text{td}_{\text{Sim}}, \text{td}_{\text{Ext}}) \leftarrow \text{SimSetup}(1^\lambda); \\ Q \leftarrow \emptyset; \\ (x^*, \pi^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}}}(\text{crs}); \\ w^* \leftarrow \text{Ext}(\text{crs}, \text{td}_{\text{Ext}}, x^*, \pi^*) \end{array} \right] \leq \text{negl}(\lambda)(\lambda).$$

Here, $\mathcal{O}_{\text{Sim}}$ denotes the simulation oracle:

$$\mathcal{O}_{\text{Sim}}(x) : Q \leftarrow Q \cup \{x\}; \text{ return } \pi \leftarrow \text{SimProve}(\text{crs}, \text{td}_{\text{Sim}}, x)$$

As noted, in our scheme, we will also require the extractor to be online (or straightline), which means that the extractor does not rewind the adversary.

3 Weighted BTX

In Construction 1, we give a construction of weighted batch threshold encryption based on the BTX scheme [3]. Given a committee of $|T|$ parties with a total weight of at least W , decryption requires $|T| + 1$ pairings per ciphertext, and the public parameters consist of one $\mathbb{G}_T$ element (the encryption key) and $(2\ell - 1)W$ $\mathbb{G}_2$ elements (the decryption keys).

The construction generalizes BTX to a weighted setting. **Setup** modifies BTX's key generation/setup by allowing different parties to have different numbers of secret shares of the underlying polynomial. It also generates per-party shares of polynomials in a different way from BTX which enables short decryption keys in the weighted setting and a simple DKG.

Enc and **PreDec** are largely the same as in BTX. **Dec** modifies BTX's decryption procedure by having one pre-decryption share correspond to multiple evaluation points/shares. We first prove the correctness of Construction 1 to give some understanding for the approach to sharing the polynomial.

Setup($1^\lambda, 1^\ell, 1^W, t, (w_j)_{j \in [N]}) \rightarrow (\text{pp}, \text{ek}, \text{dk}, (\text{sk}, \text{vk})_{j \in [N]}):$

- ▷ $\text{crs}_{\Pi_{\text{DL}}} \leftarrow \Pi_{\text{DL}}.\text{Setup}(1^\lambda).$
- ▷ Sample $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, [1]_T, p, e) \leftarrow \text{GrpGen}(1^\lambda).$
- ▷ Sample $\Omega_{[N]} \subseteq \mathbb{F}_p$ with $|\Omega_{[N]}| = W$ as an evaluation domain for secret sharing.
- ▷ Partition $\Omega_{[N]}$ into disjoint $(\Omega_j)_{j \in [N]}$ such that $|\Omega_j| = w_j$ for all $j \in [N]$.
- ▷ Let pp be $((\Omega_j)_{j \in [N]}, \text{crs}_{\Pi_{\text{DL}}}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, [1]_T, p, e).$
- ▷ Sample $z \leftarrow \mathbb{F}_p$ and a uniformly random polynomial $Z(X) \in \mathbb{F}_p[X]$ of degree at most t such that $Z(0) = z$. Set $z_\omega := Z(\omega)$ for $\omega \in \Omega_{[N]}$ and $\text{ek} := [z]_T$.
- ▷ For all $j \in [N]$, sample $q_j \leftarrow \mathbb{F}_p^*$. Let $\text{sk}_j := q_j$ denote party j 's secret key and $\text{vk}_j := (\text{vk}_{j,i} := [(q_j)^i]_2)_{i \in [\ell]}$ be their verification key.
- ▷ Set $\text{dk}_{\omega,d} := [z_\omega q_{\text{own}(\omega)}^d]_2$ for all $d \in \{-\ell, \dots, -1, 1, \dots, \ell-1\}, \omega \in \Omega_{[N]}$.
- ▷ Output $(\text{pp}, \text{ek}, (\text{sk}_j)_{j \in [N]}, (\text{vk}_j)_{j \in [N]}, \text{dk} := (\text{dk}_{\omega,d})_{d \in \{-\ell, \dots, -1, 1, \dots, \ell\}, \omega \in \Omega_{[N]}})$

Enc(ek, m) $\rightarrow \text{ct}$:

- ▷ Given $m \in \mathbb{G}_T$, sample $r \leftarrow \mathbb{F}_p$, set $\text{ct}[1] := [r]_1$ and $\text{ct}[2] := m + r \cdot \text{ek}$.
- ▷ Obtain $\pi \leftarrow \Pi_{\text{DL}}.\text{Prove}(\text{ct}[1]; r).$
- ▷ Output $\text{ct} := (\text{ct}[1], \text{ct}[2], \pi).$

PreDec($\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}$) $\rightarrow \text{sbk}$:

- ▷ Parsing $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$, if any $\Pi_{\text{DL}}.\text{Verify}(\text{ct}_i[1], \pi_i) = 0$, return $\perp$.
- ▷ Otherwise, return $\text{sbk}_j := \sum_{i=1}^\ell (q_j)^i \cdot \text{ct}_i[1] \in \mathbb{G}_1$.

Dec($\text{dk}, \{\text{sbk}_j\}_{j \in T}, (\text{ct}_i)_{i \in [\ell]}$) $\rightarrow (m_i)_{i \in [\ell]}:$

- ▷ Verify the pre-decryption keys $\{\text{sbk}_j\}_{j \in T}$ by checking $e(\text{sbk}_j, [1]_2) = \sum_{i=1}^\ell e(C_i, \text{vk}_{j,i})$ for every $j \in T$.
- ▷ Discard all pre-decryption keys which do not pass verification. Let T' be an authorized set of accepted shares. Return $\perp$ if T' no longer has sufficient weight.
- ▷ Let $\text{dk}_{\omega,0} := 0$ for all $\omega \in \Omega_{[N]}$.
- ▷ For each $i \in [\ell]$, output

$$m_i = \text{ct}_i[2] - \sum_{j \in T'} e \left(\text{sbk}_j, \sum_{\omega \in \Omega_j} \lambda_{T', \omega} \text{dk}_{\omega, -i} \right) + \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\text{ct}_k[1], \sum_{\omega \in \Omega_{T'}} \lambda_{T', \omega} \text{dk}_{\omega, k-i} \right)$$

Construction 1: Weighted BTX construction.

3.1 Correctness and robustness of Construction 1

Theorem 1 (Correctness and robustness). *Construction 1 is correct and robust.*

Proof. Correctness follows by inspection. Consider the second term in the decryption equation:

$$\begin{aligned}
& \sum_{j \in T'} e \left(\text{sbk}_j, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,-i} \right) - \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\text{ct}_k[1], \sum_{w \in \Omega'_T} \lambda_{T',w} \text{dk}_{w,k-i} \right) \\
&= \sum_{j \in T'} e \left(\sum_{k \in [\ell]} (q_j)^k \cdot [r_k]_1, \sum_{w \in \Omega_j} \lambda_{T',w} [z_w q_j^{-i}]_2 \right) - \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left([r_k]_1, \sum_{j \in T'} \sum_{w \in \Omega_j} \lambda_{T',w} [z_w (q_j)^{k-i}]_2 \right) \\
&= \left(\sum_{j \in T'} \sum_{w \in \Omega_j} \lambda_{T',w} z_w \right) [r_i]_T = r_i [z]_T = r_i \cdot \text{ek}
\end{aligned}$$

The sum simplifies to the ciphertext pad $r_i \cdot \text{ek}$, and $m_i = \text{ct}_i[2] - r_i \cdot \text{ek}$.

Next, we consider robustness. To show accurate blaming, observe that a correctly generated pre-decryption key always passes its verification check:

[Sasha: TODO: Fill in the equations here that show this]

Consistency follows from the fact that every accepted set of pre-decryption keys enables reconstructing the same one-time pad $[r_i z]_T$.

[Sasha: TODO: fill in a stronger argument here. Go from the verification equation and the honestly generated property of $C_i, vk_{j,i}$ to argue that these all represent shares of the same polynomial based on Shamir Secret Sharing.]

Accurate blaming and consistency together imply the robustness of the weighted threshold construction. $\square$

3.2 Distributed Key Generation for Construction 1

As previously mentioned, reasonably efficient distributed key generation is required in systems like Solana, where validator sets change between epochs. We describe a DKG protocol which is more efficient than a generic MPC-based setup.

Distributed key generation per epoch. The parties execute a verifiable distributed key generation protocol for a random degree- t polynomial $Z(X)$, where party j receives $[z_w := z(w)]_2$ for $w \in \Omega_j$. The DKG delivers one group element per each evaluation point owned by party j . It achieves a communication complexity of $O(N^2 + WN)$, where W is the total number of the secret shares corresponding to the total virtual weight W assigned in the scheme, and N is the total number of parties in the scheme. [Sasha: TODO: Is this a standard DKG protocol? Could you add a citation?]

Party j then samples $q_j \leftarrow \mathbb{Z}_p^*$, locally computes and publishes $\text{dk}_{w,d} := (q_j)^d [z_w]_2$ for $w \in \Omega_j$, $d \in \{-\ell, \dots, -1, 1, \dots, \ell - 1\}$. It also publishes the verification-key elements $\text{vk}_{j,i} = [(q_j)^i]_2$ for $i \in [\ell]$.

Alternate DKG approach. An alternative way to achieve DKG per epoch is as follows: each party j first publishes powers of its secret key q_j in $\mathbb{G}_2$: $[(q_j)^d]_2$ for all $d \in \{-\ell, \dots, -1, 1, \dots, \ell - 1\}$. Then, each party j samples secret shares

$z_w^{(j)}$, $w \in \Omega_{[N]}$, for some $z \leftarrow \mathbb{F}_p$, and publishes $[z^{(j)}]_T$ along with $\mathbf{dk}_{w,d,j} := z_w^{(j)} \cdot [(q_{j'})^d]_2$ for all $d \in \{-\ell, \dots, -1, 1, \dots, \ell-1\}$, $w \in \Omega'_j$, $j' \in [N]$. Finally, we set $\mathbf{dk}_{w,d} := \sum_{w,d} \mathbf{dk}_{w,d,j}$ and $[z]_T := \sum_{j \in [N]} [z^{(j)}]_T$. To ensure that the published shares can be verified, each party j can also publish $[z_w^{(j)}]_1$ along with a polynomial opening proof for w . Then, each share can be verified by checking the equation $e([z_w^{(j)}]_1, [(q_{\text{own}(w)})^d]_2) = e([1]_1, \mathbf{dk}_{w,d,j})$. **[Sasha: I'm confident that the first approach works, a little confused by the second approach.]** **[Aditi: maybe we also check that $e(\prod_{w \in \Omega_j} [z_w^{(j)}]_1, [1]_2) = [z^{(j)}]_T$]**

3.3 Optimizations to Construction 1

Next, prior schemes including BTX [3] and BEAT⁺⁺ [1], accelerate their decryption procedures using FFTs. Our scheme also supports this structure. During the decryption of the ciphertexts, the first terms

$$e\left(\mathbf{sbk}_j, \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,-i}\right)$$

can be computed via a multi-pairing of arity $|T|$. The remaining term

$$\sum_{\substack{k \in [\ell] \\ k \neq i}} e\left(\mathbf{ct}_k[1], \sum_{w \in \Omega_T} \lambda_{T,w} \mathbf{dk}_{w,k-i}\right)$$

forms a middle product. After zero-padding the two input sequences to $\ell = 2^{\lceil \log_2(2\ell) \rceil}$, we can compute a group FFT in $\mathbb{G}_1$, a group FFT in $\mathbb{G}_2$, do m coordinate-wise pairings, and finally do an inverse FFT in $\mathbb{G}_T$. Thus, all ciphertexts can be recovered using ℓ MultiPairing operations of size $|T|$ and m pairings (i.e., $|T| + 1$ pairings per ciphertext). Note that all submitted pre-decryption keys can also be batch verified.

[Sasha: TODO: Do we really have to zero-pad? Does this satisfy the “no padding” feature of BTX?]

3.4 Assumption and CCA security

We first state the security assumption used by our scheme. It is a variation of the decisional power Diffie–Hellman assumption used by BTX [3, Assumption 4.1].

Assumption 2 (Weighted missing-term DH). *Fix $(\Omega_j)_{j \in [N]}, t, \ell$. The challenger generates $Z, z, (q_j)_j$ and all algebraic public values **[Sasha: TODO: write out the algebraic public values which aren't independent of this assumption.]** exactly as in Construction 1 (the independent NIZK CRS is omitted), and defines*

$$H_{j,i} = [q_j^i]_1 \quad (j \in [N], i \in [\ell]).$$

It first gives $(\text{ek}, \text{dk}, (H_{j,i})_{j,i})$ to the distinguisher, which returns an unauthorized set $\mathcal{C} \subseteq [N]$ and state. The challenger then samples $r, u \leftarrow \mathbb{F}_p$, gives $(q_j)_{j \in \mathcal{C}}$ and either

$$(X, Y) = ([r]_1, [rz]_T) \quad \text{or} \quad (X, Y) = ([r]_1, [u]_T),$$

and receives one decision bit. The advantage is the absolute difference between the two acceptance probabilities. We say WMT-DH holds if this advantage is negligible for all PPT distinguishers and all polynomially bounded parameters.

Theorem 3 (Static batch CCA security). *If Assumption 2 holds and Π_{DL} is complete, zero knowledge, and online simulation extractable for R_{DL} , then Construction 1 is static WB-IND-CCA secure.*

Proof. Let $\mathcal{A}$ attack Definition 3. We build a WMT-DH distinguisher $\mathcal{B}$. On receiving $(\text{ek}, \text{dk}, (H_{j,i})_{j,i})$, it runs $(\text{crs}, \text{td}_{\text{sim}}, \text{td}_{\text{ext}}) \leftarrow \Pi_{\text{DL}}.\text{SimSetup}(1^\lambda)$ and gives $\mathcal{A}$ the real public values and simulated CRS. When $\mathcal{A}$ returns its unauthorized set $\mathcal{C}$, $\mathcal{B}$ forwards it to the assumption challenger and gives the returned $(q_j)_{j \in \mathcal{C}}$ to $\mathcal{A}$. It also receives the assumption challenge (X, Y) .

For every accepted ciphertext (C_i, E_i, π_i) in a decryption query, $\mathcal{B}$ invokes the online extractor on the fresh statement (C_i, E_i) , obtaining r_i with $C_i = [r_i]_1$. It returns, for each honest $j \notin \mathcal{C}$,

$$\sigma_j = \sum_{i=1}^{\ell} r_i H_{j,i},$$

which is exactly the honest pre-decryption share. The probability of an extraction failure on a fresh accepting statement is negligible by simulation extractability. An exact challenge ciphertext is rejected by the game. If only the proof is changed, pair-fresh extraction applies; if the first component is reused with a different second component, the statement itself is fresh. [\[Sasha: These previous three sentences are super AI slop.\]](#)

When $\mathcal{A}$ submits m_0, m_1 , $\mathcal{B}$ chooses $b \leftarrow \{0, 1\}$ and return

$$\text{ct}^* = (X, m_b + Y, \Pi_{\text{DL}}.\text{SimProve}(\text{td}_{\text{sim}}, (X, m_b + Y))).$$

If $(X, Y) = ([r]_1, [rz]_T)$, this is the real game up to zero knowledge and negligible extraction failure. If Y is uniform, the second component is a perfect one-time pad independent of b ; hence $\Pr[b' = b] = 1/2$. Using $b' = b$ as its decision, $\mathcal{B}$'s distinguishing advantage is $\text{Adv}_{\text{WBT}, \mathcal{A}}^{\text{WB-IND-CCA}} - \text{negl}(\lambda)$. $\square$

Lemma 1 (Generic bilinear group hardness). *Assumption 2 holds in the generic bilinear group model, up to the a standard $O(Q^2 \Delta/p)$ collision term for Q oracle operations and polynomially-bounded formal degree Δ .*

Proof sketch. Represent generic encodings by Laurent polynomials³ in independent symbols

$$r, z, a_1, \dots, a_t, (q_j)_{j \in [N]},$$

³ Laurent polynomials are polynomials which can have monomials of both positive and negative degree.

where $Z(X) = z + \sum_{k=1}^t a_k X^k$. The only $\mathbb{G}_2$ terms containing an honest evaluation for Shamir secret sharing are $Z(w)q_{\text{own}(w)}^d$ with $d \neq 0$. Pairing them with $[r]_1$ leaves the nonzero Laurent monomial in the independent honest-party variable $q_{\text{own}(w)}$, so no linear combination yields rz . Pairing a simulation handle q_j^i with a term of exponent $-i$ can remove that monomial, but produces a target-group term without the factor r , and target-group elements cannot be paired again.

For corrupted parties, the revealed q_j 's let the adversary obtain terms $rZ(w)$ at the corrupted virtual points. Since $|\Omega_{\mathcal{C}}| \leq t$, the constant coefficient z is not in the linear span of these evaluations of a uniformly random degree- t polynomial. Thus rz is absent from the formal span of every generically computable target-group element. Replacing it by an independent symbol is therefore information-theoretically hidden until a formal-expression collision occurs. Clearing Laurent denominators and applying Schwartz–Zippel gives the stated collision bound. This is the same missing-power argument underlying generic hardness of decisional nDH, with the independence of Shamir secret shares accounting for corrupted virtual points. The reasoning applies to every realized unauthorized $\mathcal{C}$, so the two-stage choice of the corruption set does not change the bound. **[Sasha: TODO: This argument needs to be formalized quite a bit.]** $\square$

3.5 Cost analysis for Construction 1

In Table 1, we give the storage/communication costs for the weighted BTX scheme. In Table 2, we give computation costs for the weighted BTX scheme.

4 Partial fraction-based Weighted BTE

Our second construction, presented in Construction 2, adapts the partial fraction-based batch threshold encryption scheme of [9]. For $a \in \mathbb{F}_p$, define $p_a(X) := \frac{1}{X+a}$. Given an authorized committee of $|T|$ parties with total weight W , decryption requires $|T| + 2$ pairings per ciphertext, and the public parameters consist of one $\mathbb{G}_T$ element (the encryption key) and $((2\ell)W + 1)|\mathbb{G}_2|$ elements (the decryption keys). We fix distinct public $\alpha_1, \dots, \alpha_\ell \in \mathbb{F}_p \setminus \{0, -1\}$.

Construction 2 adapts the construction of [9] by assigning secret shares to each party in a different way from the prior work.

4.1 Correctness and robustness of Construction 2

Theorem 4 (Correctness and robustness). *Construction 2 is correct and satisfies robustness.*

Table 1: Storage and communication costs for Construction 1. N is the number of real parties, W is the total weight, $\tau = |T|$ is the number of parties in the accepted set, and ℓ is the batch size. We exclude costs related to the NIZK.

Quantity	Size
Encryption key $\mathbf{ek}$	$ \mathbb{G}_T $
Party secret key $\mathbf{sk}_j$	$ \mathbb{F}_p $
Party verification key $\mathbf{vk}_j$	$\ell \mathbb{G}_2 $
All parties' verification keys	$N\ell \mathbb{G}_2 $
Core decryption material $(D_{w,d})$	$(2\ell - 1)W \mathbb{G}_2 $
Public decryption key $\mathbf{dk}$, including all verification keys	$((2\ell - 1)W + N\ell) \mathbb{G}_2 $
Non-NIZK CRS / total public parameters $(\mathbf{ek}, \mathbf{dk})$	$((2\ell - 1)W + N\ell) \mathbb{G}_2 + \mathbb{G}_T $
Ciphertext, excluding π_{DL}	$ \mathbb{G}_1 + \mathbb{G}_T $
Padded batch of ciphertexts, excluding proofs	$\ell \mathbb{G}_1 + \ell \mathbb{G}_T $
One party's pre-decryption σ_j	$ \mathbb{G}_1 $
Pre-decryptions from an accepted set T	$\tau \mathbb{G}_1 $
Decrypted message batch	$\ell \mathbb{G}_T $

Table 2: Computation costs for Construction 1. For $a \in \{1, 2, T\}$, $\mathbf{SM}_a$ is a scalar multiplication in $\mathbb{G}_a$ and $\mathbf{MSM}_a(q)$ is a multi-scalar multiplication of size q in $\mathbb{G}_a$. Group additions are not counted. $\mathbf{P}$ denotes a pairing, $\mathbf{MP}(q)$ denotes a multi-pairing of size q , and $\mathbf{FFT}_a(q)/\mathbf{IFFT}_a(q)$ denote size q FFT's in $\mathbb{G}_a$. We set $m = 2^{\lceil \log_2(2\ell) \rceil}$.

Operation	Cost
Enc, one ciphertext	$1\mathbf{SM}_1 + 1\mathbf{SM}_T$
PreDec $_j$, one padded batch	$1\mathbf{MSM}_1(\ell)$
Verify one pre-decryption σ_j	$1\mathbf{MP}(\ell + 1)$
Dec, naive cross-term evaluation	$\ell\mathbf{MP}(\tau + \ell - 1) + \sum_{j \in T} \ell\mathbf{MSM}_2(w_j) + (\ell - 1)\mathbf{MSM}_2(W_T)$
Dec, FFT-based middle-product evaluation	$\ell\mathbf{MP}(\tau) + m\mathbf{P} + \mathbf{FFT}_1(m) + \sum_{j \in T} \ell\mathbf{MSM}_2(w_j) + (\ell - 1)\mathbf{MSM}_2(W_T) + \mathbf{FFT}_2(m) + \mathbf{IFFT}_T(m)$

Proof. Observe that

$$\begin{aligned}
& e \left(\mathbf{sbk}_j - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{\mathbf{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,i}[1] \right) \\
&= e \left(\sum_{k \in [\ell]} g_{j,k}(x) \cdot [r_k]_1 - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{[r_k]_1}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} [z_w g_{\text{own}(w),i}(x)]_2 \right) \\
&= e \left(g_{j,i}(x)^2 \cdot [r_i]_1, \sum_{w \in \Omega_j} \lambda_{T,w} \cdot [z_w]_2 \right) \\
&\quad - \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(g_{j,i}(x) \left(g_{j,k}(x) \cdot [r_k]_1 - \frac{[r_k]_1}{\alpha_k - \alpha_i} \right), \sum_{w \in \Omega_j} \lambda_{T,w} [z_w]_2 \right)
\end{aligned}$$

and

$$\sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,k}[1] \right) = \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(g_{j,k}(x) \frac{[r_k]_1}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} [z_w]_2 \right)$$

Therefore, adding the two terms cancels the ‘−’ term of the first expression and the second expression, yielding

$$\sum_{j \in T} L_{j,i} = \sum_{j \in T} e \left(g_{j,i}(x)^2 \cdot [r_i]_1, \sum_{w \in \Omega_j} \lambda_{T,w} \cdot [z_w]_2 \right) = \sum_{j \in T} \sum_{w \in \Omega_j} [g_{j,i}(x)^2 r_i \lambda_{T,w} z_w]_T$$

Rest of the correctness proof follows from the observation that

$$\begin{aligned} e \left(\text{ct}_i[1], \sum_{w \in \Omega_T} \lambda_{T,w} \mathbf{dk}_{w,i}[2] \right) &= \left(\frac{p_{-1}(x)}{\alpha_i + 1} + \frac{p_0(x)}{\alpha_i} \right) [r_i z]_T \\ &\quad + \sum_{j \in T} \sum_{w \in \Omega_j} [g_{j,i}(X)^2 r_i \lambda_{T,w} z_w]_T, \end{aligned}$$

where the latter term cancel with $\sum_{j \in T} L_{j,i}$ in the decryption expression.

For accurate blaming, again, observe that a correctly generated pre-decryption key always passes its verification check. Consistency follows from the fact that every accepted set of pre-decryption keys enables reconstructing the same one-time pads

$$\left(\frac{p_{-1}(x)}{\alpha_i + 1} + \frac{p_0(x)}{\alpha_i} \right) [r_i z]_T.$$

Accurate blaming and consistency together imply the robustness of the weighted threshold construction. $\square$

4.2 Distributed Key Generation for Construction 2

4.3 Optimizations to Construction 2

Aggregated decryption. During the decryption of the ciphertexts, the first terms

$$e \left(\text{sbk}_j - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,i}[1] \right)$$

can again be computed via a multi-pairing of arity $|T|$. For the second terms

$$\sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,k}[1] \right) = \frac{1}{\alpha_k - \alpha_i} \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\text{ct}_k[1], \sum_{w \in \Omega_j} \lambda_{T,w} \mathbf{dk}_{w,k}[1] \right),$$

observe that we can compute

$$e\left(\text{ct}_k[1], \sum_{w \in \Omega_j} \lambda_{T,w} \text{dk}_{w,k}[1]\right)$$

for each $k \in [\ell]$ once, and re-use these terms during decryption. Consequently, decryption uses exactly ℓ multi-pairings of size $|T|$ and 2ℓ pairings, i.e., $|T| + 1$ pairings per ciphertext. Note that all submitted pre-decryption keys can also be batch verified.

4.4 Assumption and CCA Security

4.5 Analysis of Construction 2

Lemma 2 (Generic bilinear group hardness). *Assumption ?? holds in the generic bilinear group model, up to an $O(Q^2 \Delta/p)$ collision term.*

Proof sketch. Work with rational functions in the independent symbols $r, x, z, a_1, \dots, a_t, (\rho_j)_j$, with $Z(X) = z + \sum_{k=1}^t a_k X^k$. Any generically computed target-group term that contains r arises by pairing $[r]_1$ with a public $\mathbb{G}_2$ encoding. The verification keys contribute terms $rg_{j,i}(x)$, which contain no variable related to Shamir secret sharing. For an honest party j , every $Z(w)u_{j,i}(x)$ has a double pole at $x = -\rho_j - \alpha_i$, unique to (j, i) . The target $rzp_0(x)$ has no such pole, so all coefficients of honest-party $rZ(w)u_{j,i}(x)$ terms must vanish. At the remaining simple pole, the coefficient lies in the span of 1 and the relevant Shamir secret sharing evaluations. It must vanish because a nonzero linear form in a random degree- t Shamir secret sharing polynomial cannot equal a fixed constant. Thus neither the verification-key terms nor $rZ(w)g_{j,i}(x)$ can contribute to the target.

The global element V yields $rz(p_{-1}(x) + p_0(x))$. Eliminating its pole at $x = 1$ using corrupted-party material would require expressing z as a linear combination of $(Z(w))_{w \in \Omega_C}$. This is impossible because $|\Omega_C| \leq t$ evaluations do not determine the constant coefficient of a random degree- t polynomial. Thus the coefficient of the global term must also vanish, and $rzp_0(x)$ is outside the formal bilinear span.

The handles $H_{j,i}$ do not alter this conclusion: pairing one with public material creates a target-group rational function without the factor r , and target-group encodings cannot be paired again. After clearing all denominators, Schwartz–Zippel bounds accidental equalities. This is the same pole-order argument used for generic hardness of q -QuadRankDef [9, Lemma 7], augmented by the independence (in the sense of Shamir secret sharing) at the corrupted virtual points and by distinct party shifts. It holds for every realized unauthorized $\mathcal{C}$, including the two-stage choice in Assumption ??. $\square$

4.6 Cost analysis for Construction 2

In Table 3, we give the storage and computational costs for the partial fraction-based weighted BTE scheme.

For the storage costs, N is the number of real parties, $W = \sum_{j=1}^N w_j$ is the total virtual weight, $\tau = |T|$ is the number of parties in the accepted set, and ℓ is the padded batch size. The NIZK CRS, ciphertext proofs, public bilinear-group description, and fixed generators are excluded.

For the computational costs and $a \in \{1, 2, T\}$, SM_a denotes one scalar multiplication in $\mathbb{G}_a$, $\text{MSM}_a(q)$ denotes one size- q multi-scalar multiplication in $\mathbb{G}_a$, and Add_a denotes one group addition or subtraction in $\mathbb{G}_a$. The suffixes 1, 2, and T refer respectively to $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$. An ordinary pairing P is listed when its individual $\mathbb{G}_T$ output is required. A q -term multi-pairing $\text{MP}(q)$ computes a product of q pairings using q Miller-loop inputs and one shared final exponentiation. $\text{CauchyFFT}_a(\ell)$ denotes collective evaluation of the Cauchy sums using $O(\ell \log \ell)$ operations in $\mathbb{G}_a$. Here $W_T = \sum_{j \in T} w_j$. The decryption costs use the corrected formula for R_i . Setup and the internal cost of Π_{DL} are excluded.

5 Experiments

We next report performance results from our experiments. All experiments were run on an Apple MacBook Pro with an M4 processor and 24 GB of RAM. We implemented our experiments in Rust and used the `blst` library’s [22] implementation of fields, curves and pairings. Code for our experiments is available in [5].

We benchmarked the schemes with an authorization threshold of $1/2$. We chose the distribution of weights to approximate Solana’s share distribution [6]. We used Aptos’ approach [7] to compute the share assignments and total required numbers of shares. For our experiments, we chose total weights of 764; 1,580; 3,063; and 6,489 corresponding to different qualities of approximation. We describe the parameter choices and Aptos’ algorithm in greater depth in Appendix A. To contextualize the performance numbers in this paper, our target performance numbers were for batch sizes of 32 ciphertexts. These parameters were based on discussions with the Solana developers [20] about their performance needs for such a scheme ⁴.

5.1 Performance results for Construction 1

In Table 4, we give performance numbers for our weighted version of the BTX scheme. We hold the batch size ℓ fixed at 32, and vary the total number of shares. As previously mentioned, the share counts were computed based on an adaptation of Aptos’ algorithm. We run these experiments on 12 threads.

We also reimplemented BTX. Our implementation was $\sim 50\%$ slower than theirs. We note that BTX has an optimized AVX-512 based implementation of various operations, which likely accounts for the performance overheads. Using analogous

⁴ Solana includes more than 500 ciphertexts per block, but in the proposed MCP architecture, shards of ~ 30 blocks are handled separately. TODO: Fill in the correct explanation.

5.2 Performance results for Construction 2

In Table 5, we .

We also reimplemented the basic partial fraction-based batch threshold encryption scheme of [9].

5.3 Comparison to prior work

We compare against a few prior works.

6 Related work

We discuss the main relevant lines of prior research to this work.

Encrypted mempools/MEV. Public blockchain mempools reveal pending transactions before their ordering is finalized, enabling front-running and other forms of maximal extractable value (MEV) [15]. Encrypted mempools seek to mitigate these attacks by hiding transaction contents until their order has been fixed. This setting motivates threshold encryption schemes which remain efficient for large batches of transactions and whose access structures reflect each validator’s stake [17].

Batch- and epoch-labelled BTE. Choudhuri et al. [13] introduced the idea of batch threshold encryption (BTE), which lets each party in a threshold encryption scheme publish a short pre-decryption share to decrypt an entire selected batch of ciphertexts while ciphertexts outside that batch remain private. Subsequent works improved on this work by supporting reusable setups or building related batched identity- and tag-based schemes [14,4,21]. These constructions still associate every ciphertext with some sort of batch-specific information, so a ciphertext omitted from its intended batch may become “stranded”.

Epochless, index-dependent BTE. BEAT-MEV [12] removes batch labels and allows ciphertexts to be reused across batches. However, its encryption requires an index from a fixed public domain, and a decryptable batch must contain at most one ciphertext for each index. Dealing with index collisions requires additional communication or leads to issues with censorship resistance. Subsequent works improve on this approach [11,1].

Weighted threshold cryptography. Proof-of-stake systems motivate cryptographic schemes with weighted access structures, since the influence of a validator is determined by its stake. Ferveo [8] supports (non batched) weighted threshold decryption while compressing the virtual decryption shares for one ciphertext into a single group element per party. Agarwal et al. [2] give the first weighted BTE scheme where a party’s pre-decryption communication is independent of its assigned weight, and reduce decryption from quadratic to quasilinear complexity, though it still its encryption is still index-dependent.

Epochless, index-free BTE. Recent works construct pairing-based Batch Threshold Encryption schemes without indices or epoch-specific labels. BTX [3]

and Policharla [19] give the same BTE construction with short ciphertexts and quasilinear decryption. Concurrently, Boneh et al. [9] adapt partial fraction techniques [18] to construct an epochless and index-free BTE scheme. Both schemes achieve similar properties, though the BTX construction is concretely more efficient. We use these schemes as the starting points for Construction 1 and Construction 2 respectively. We extend them to support weighted access structures and a simple DKG procedure.

7 Conclusion

We introduce two constructions for weighted batch threshold encryption. Starting from the BTX construction of Agarwal et al. and the partial fraction construction of Boneh et al., we build two schemes where each party releases only one group element for an entire ciphertext batch as part of decryption, independent of its assigned weight. Together, they are the first weighted batch threshold encryption schemes with batch-independent, index-free ciphertexts. For the more efficient BTX-based construction, we additionally provide a specialized distributed key-generation protocol that avoids a general-purpose MPC setup. Our Rust implementation further demonstrates that these techniques are practical at parameter sizes relevant to encrypted mempools. For remaining future work:

[Sasha: Finish the discussion of limitations and future work when the paper is closer to completion.]

References

1. Agarwal, A., Babel, K., Das, S., Gilkalaye, B.P., Mondal, A., Pinkas, B., Rindal, P., Yadav, A.: Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Paper 2025/2115 (2025), <https://eprint.iacr.org/2025/2115>
2. Agarwal, A., Babel, K., Das, S., Gilkalaye, B.P., Mondal, A., Pinkas, B., Rindal, P., Yadav, A.: Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Report 2025/2115 (2025), <https://eprint.iacr.org/2025/2115>
3. Agarwal, A., Das, S., Gilkalaye, B.P., Rindal, P., Shoup, V.: BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Paper 2026/754 (2026), <https://eprint.iacr.org/2026/754>
4. Agarwal, A., Fernando, R., Pinkas, B.: Efficiently-thresholdizable batched identity based encryption, with applications. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part III. LNCS, vol. 16002, pp. 69–100. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01881-6_3
5. Anonymous: Weighted BTE. https://anonymous.4open.science/r/weighted_bte (2026)
6. Anza: Scheduler War. <https://schedulerwar.vercel.app/> (2026), accessed: 2026-08-06
7. Aptos Foundation: real_dkg/rounding/mod.rs. https://github.com/aptos-labs/aptos-core/blob/main/types/src/dkg/real_dkg/rounding/mod.rs

8. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898 (2022), <https://eprint.iacr.org/2022/898>
9. Boneh, D., Nema, R., Roy, A., Tas, E.N.: Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive, Paper 2026/674 (2026), <https://eprint.iacr.org/2026/674>
10. Boneh, D., Shoup, V.: A graduate course in applied cryptography. <https://toc.cryptobook.us/book.pdf> (2023)
11. Bormet, J., Choudhuri, A.R., Faust, S., Garg, S., Othman, H., Policharla, G.V., Qu, Z., Wang, M.: BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Report 2025/1419 (2025), <https://eprint.iacr.org/2025/1419>
12. Bormet, J., Faust, S., Othman, H., Qu, Z.: BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In: Bauer, L., Pellegrino, G. (eds.) USENIX Security 2025. pp. 3457–3476. USENIX Association (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/bormet>
13. Choudhuri, A.R., Garg, S., Piet, J., Policharla, G.V.: Mempool privacy via batched threshold encryption: Attacks and defenses. In: Balzarotti, D., Xu, W. (eds.) USENIX Security 2024. USENIX Association (Aug 2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>
14. Choudhuri, A.R., Garg, S., Policharla, G.V., Wang, M.: Practical mempool privacy via one-time setup batched threshold encryption. In: Bauer, L., Pellegrino, G. (eds.) USENIX Security 2025. pp. 3477–3495. USENIX Association (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/choudhuri>
15. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: 2020 IEEE Symposium on Security and Privacy. pp. 910–927. IEEE Computer Society Press (May 2020). <https://doi.org/10.1109/SP40000.2020.00040>
16. Das, S., Pinkas, B., Tomescu, A., Xiang, Z.: Distributed randomness using weighted VRFs. Cryptology ePrint Archive, Report 2024/198 (2024), <https://eprint.iacr.org/2024/198>
17. Dziembowski, S., Faust, S., Luhn, J.: Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Report 2024/1981 (2024), <https://eprint.iacr.org/2024/1981>
18. Jutla, C.S., Nema, R., Roy, A.: Partial fraction techniques for cryptography. In: Daemen, J., Thomé, E. (eds.) EUROCRYPT 2026, Part V. LNCS, vol. 16545, pp. 423–453. Springer, Cham (May 2026). https://doi.org/10.1007/978-3-032-25330-9_15
19. Policharla, G.V.: A simple batched threshold encryption scheme. Cryptology ePrint Archive, Paper 2026/760 (2026), <https://eprint.iacr.org/2026/760>
20. Solana core developers: Personal communication. Private communication with the authors (2026)
21. Suegami, S., Ashizawa, S., Shibano, K.: Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762 (2024), <https://eprint.iacr.org/2024/762>
22. Supranational: blst: Multilingual BLS12-381 signature library. Software repository (2020), <https://github.com/supranational/blst>

A Choosing total weights for the Solana chain

For our experiments, we computed per-party weights and total weight based on a snapshot of the stake distribution among Solana’s validators obtained from [6]. At the time of writing, there were 862 validator accounts with active stake, totaling about 478.4 million SOL.

Aptos implements a weighted threshold cryptographic scheme based on [16]. They implement an algorithm for assigning integer weights to parties based on stake percentages in [7]. We use Aptos’ algorithm to compute the per-party weights approximating Solana’s share distribution, choosing it because this is an algorithm with real-world usage. However, there are multiple, equally valid approaches.

To explain Aptos’ algorithm, assigning integer weights approximating each party’s stake necessarily leads to some approximation error from using integer share counts. Slightly reinterpreting Aptos’ algorithm, it takes a target threshold T and acceptable deviation ε . The algorithm computes a minimum share count such that no decryption committee with true stake fraction less than $T - \varepsilon$ is authorized, and that any committee with true stake fraction $T + \varepsilon$ is unauthorized. The algorithm then rounds each party’s fraction of the stake to determine the number of shares to assign to each party.

In Table 6, for $T = 1/2$ and varying ε , we show W , the minimal total weight required to achieve error ε , as well as some statistics for each resulting assignment of weights. Specifically, we report the actual threshold q , the minimum weight required to decrypt ciphertexts. We also specify the number of validators assigned nonzero weights for each value of ε and the percentage of stake assigned to validators who receive zero shares. We also measure the quality of approximations. Letting $\hat{p}_j$ be w_j/W be validator j ’s approximate stake fraction and p_j be their true stake fraction, we report statistics on each party’s absolute approximation error $|\hat{p}_j - p_j|$, as well as their relative error $|\hat{p}_j - p_j|/p_j$. Validators receiving zero weight are included in these statistics.

Table 6: Aptos-style share approximations to the Solana active-stake distribution for target threshold $T = 2/3$ and varying allowed deviations ε . Average stake per virtual share is rounded to the nearest SOL.

Metric	Allowed deviation ε					
	1/4	1/8	1/16	1/32	1/64	1/128
Computed total weight W	764	764	1,580	3,063	6,489	12,460
Minimum reconstruction weight q	404	512	1,060	2,044	4,327	8,308
$(T + \varepsilon)$ reconstruction boundary (%)	63.129186	75.658188	72.884133	69.791029	68.227607	67.447787
Validators with nonzero weights	234	234	419	598	688	729
Stake (%) held by validators with zero shares	13.3965	13.3965	5.5137	1.4216	0.3656	0.1176
Mean absolute error (pp)	0.032983	0.032983	0.015630	0.007260	0.003622	0.001814
Median absolute error (pp)	0.023706	0.023706	0.016225	0.007337	0.003650	0.001768
Maximum absolute error (pp)	0.452110	0.452110	0.078842	0.019015	0.007689	0.004087
Median relative error (%)	100.00	100.00	100.00	25.98	13.14	5.90

We choose the weight distributions with $\varepsilon = 1/8, 1/16, 1/32, 1/64$ for our experiments. The resulting total share counts are comparable to those used by [16]’s experiments. Furthermore, at $\varepsilon = 1/32, 1/64$, the high and low end reconstruction thresholds come within a few percentage points of the true threshold, and the various error statistics drop to a few basis points. Our reimplementation of Aptos’ share assignment rule and generated allocations are available in the `scripts` directory at [5].

Setup($1^\lambda, 1^\ell, (\Omega_j)_{j \in [N]}, 1^t, 1^W$) $\rightarrow$ (ek, sk, vk, dk):

- ▷ $\text{crs}_{\Pi_{\text{DL}}} \leftarrow \Pi_{\text{DL}}.\text{Setup}(1^\lambda)$.
- ▷ Sample $x \leftarrow \mathbb{F}_p \setminus \{0, 1\}$, $z \leftarrow \mathbb{F}_p$, and a uniformly-random polynomial $Z(X) \in \mathbb{F}_p[X]$ of degree t such that $Z(0) = z$. Set $z_w := Z(w)$ for $w \in \Omega_{[N]}$.
- ▷ For every $j \in [N]$, sample a *hidden* shift ρ_j such that the denominators of all fractions involved in the scheme are nonzero, and all values $\rho_j + \alpha_i$ for $(j, i) \in [N] \times [\ell]$, are distinct. Define

$$g_{j,i}(X) := p_{\rho_j + \alpha_i}(X) \quad u_{j,i}(X) := g_{j,i}(X)^2 + \frac{p_{-1}(X)}{\alpha_i + 1} + \frac{p_0(X)}{\alpha_i}$$

for all $j \in [N]$ and $i \in [\ell]$.

- ▷ Let $\text{ek} := [zp_0(x)]_T$, and $\text{sk}_j := (g_{j,i}(x))_{i \in [\ell]}$, $\text{vk}_j := (\text{vk}_{j,i} := [g_{j,i}(x)]_2)_{i \in [\ell]}$ denote party j 's secret and verification keys respectively.
- ▷ Define $\text{dk}[0] := [z(p_{-1}(x) + p_0(x))]_2$, and for all $w \in \Omega_{[N]}$ and $i \in [\ell]$,

$$\text{dk}_{w,i}[1] := [z_w g_{\text{own}(w),i}(x)]_2 \quad \text{and} \quad \text{dk}_{w,i}[2] := [z_w u_{\text{own}(w),i}(x)]_2$$

- ▷ Output

$$\left(\text{ek}, (\text{sk}_j)_{j \in [N]}, (\text{vk}_j)_{j \in [N]}, \text{dk} := \left(\text{dk}[0], (\text{dk}_{w,i}[1], \text{dk}_{w,i}[2])_{w \in \Omega_{[N]}, i \in [\ell]} \right) \right).$$

Enc(ek, m) $\rightarrow$ ct:

- ▷ Given $m \in \mathbb{G}_T$, sample $r \leftarrow \mathbb{F}_p$, and set $\text{ct}[1] := [r]_1$ and $\text{ct}[2] := m + r \cdot \text{ek}$.
- ▷ Obtain $\pi \leftarrow \Pi_{\text{DL}}.\text{Prove}(\text{ct}[1]; r)$.
- ▷ Output $\text{ct} := (\text{ct}[1], \text{ct}[2], \pi)$.

PreDec(sk_j, (ct_i)_{i ∈ [ℓ]}) $\rightarrow$ sbk:

- ▷ Parse $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$ for every $i \in [\ell]$. If $\Pi_{\text{DL}}.\text{Verify}(\text{ct}_i[1], \pi_i) = 0$ for any $i \in [\ell]$, return $\perp$.
- ▷ Otherwise, return $\text{sbk}_j := \sum_{i=1}^{\ell} g_{j,i}(x) \cdot \text{ct}_i[1] \in \mathbb{G}_1$.

Dec(dk, sbk, (ct_i)_{i ∈ [ℓ]}) $\rightarrow$ (m_i)_{i ∈ [ℓ]}:

- ▷ Verify the pre-decryption keys $\{\text{sbk}_j\}_{j \in T}$ by checking

$$e(\text{sbk}_j, [1]_2) = \sum_{i=1}^{\ell} e(\text{ct}_i[1], \text{vk}_{j,i}).$$

for every $j \in T$

- ▷ Discard all pre-decryption keys which do not pass verification. Let T' be an authorized set of accepted pre-decryption keys. Return $\perp$ if T' no longer has sufficient weight.
- ▷ For every $j \in T'$, $i \in [\ell]$, compute

$$\begin{aligned} L_{j,i} := & e \left(\text{sbk}_j - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,i}[1] \right) \\ & + \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left(\frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,k}[1] \right) \end{aligned}$$

- ▷ For every $i \in [\ell]$, output $m_i = \text{ct}_i[2] - R_i$, where

$$R_i := \alpha_i \left((\alpha_i + 1) \left(e \left(\text{ct}_i[1], \sum_{w \in \Omega_{T'}} \lambda_{T',w} \text{dk}_{w,i}[2] \right) - \sum_{j \in T'} L_{j,i} \right) - e(\text{ct}_i[1], \text{dk}[0]) \right)$$

Table 3: Storage and online algebraic costs for the partial fraction weighted BTE construction.

Storage costs	
Quantity	Size
Encryption key $\mathbf{ek}$	$ \mathbb{G}_T $
Party secret key $\mathbf{sk}_j$	$\ell \mathbb{F}_p $
Party verification key $\mathbf{vk}_j$	$\ell \mathbb{G}_2 $
All parties' verification keys	$N\ell \mathbb{G}_2 $
Core decryption material $(D_{w,i}, U_{w,i}, V)$	$(2W\ell + 1) \mathbb{G}_2 $
Public decryption key $\mathbf{dk}$, including all verification keys	$((2W + N)\ell + 1) \mathbb{G}_2 $
Non-NIZK CRS / total public parameters $(\mathbf{ek}, \mathbf{dk})$	$((2W + N)\ell + 1) \mathbb{G}_2 + \mathbb{G}_T $
Ciphertext, excluding π_{DL}	$ \mathbb{G}_1 + \mathbb{G}_T $
Padded batch of ciphertexts, excluding proofs	$\ell \mathbb{G}_1 + \ell \mathbb{G}_T $
One party's pre-decryption σ_j	$ \mathbb{G}_1 $
Pre-decryptions from an accepted set T	$\tau \mathbb{G}_1 $
Decrypted message batch	$\ell \mathbb{G}_T $
Computational costs	
Operation	Cost
Enc, one ciphertext	$1 \text{ SM}_1 + 1 \text{ SM}_T + 1 \text{ Add}_T$
PreDec $_j$, one padded batch	$1 \text{ MSM}_1(\ell)$
Verify one submitted pre-decryption σ_j	$1 \text{ MP}(\ell + 1)$
Dec, cached displayed form	$\ell(\tau + 1)\text{P} + \ell \text{ MP}(\tau + 1)$ $\ell \text{ MSM}_1(\ell - 1) + \tau \ell \text{ Add}_1$ $\sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T)$ $\tau \ell \text{ MSM}_T(\ell - 1) + \ell \text{ MSM}_T(2) + \ell(\tau + 1) \text{ Add}_T$
Dec, cached and bilinearly aggregated	$2\ell \text{P} + \ell \text{ MP}(\tau)$ $\ell \text{ MSM}_1(\ell - 1) + \tau \ell \text{ Add}_1$ $\sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T) + \ell \text{ MSM}_2(2) + \ell(\tau - 1) \text{ Add}_2$ $\ell \text{ MSM}_T(\ell - 1) + \ell \text{ SM}_T + 3\ell \text{ Add}_T$
Dec, cached/aggregated pairings and FFT-accelerated Cauchy sums	$2\ell \text{P} + \ell \text{ MP}(\tau)$ $\text{CauchyFFT}_1(\ell) + \tau \ell \text{ Add}_1$ $\sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T) + \ell \text{ MSM}_2(2) + \ell(\tau - 1) \text{ Add}_2$ $\text{CauchyFFT}_T(\ell) + \ell \text{ SM}_T + 3\ell \text{ Add}_T$

	partialDecrypt(B) (ms/item)			precompute(B) (ms/item)			ctxtCheck(B) (ms/item)	open(B) (ms/item)		
W	PFE	BTX	BEAT++	PFE	BTX	BEAT++	BTX	PFE	BTX	BEAT++
765	0.035	0.035	0.039	0.963	0.644	0.642	0.097	0.721	0.171	0.171
1,580	0.029	0.029	0.039	1.120	0.722	0.722	0.097	0.721	0.171	0.172
3,063	0.025	0.025	0.039	1.278	0.801	0.800	0.098	0.721	0.171	0.171
256	0.022	0.022	0.039	1.436	0.880	0.880	0.098	0.722	0.171	0.171

Table 4: Mean running times of the batch-size-dependent phases, together with the isolated ciphertext-proof verification cost $\text{ctxtCheck}(B)$.

	partialDecrypt(B) (ms/item)			precompute(B) (ms/item)			ctxtCheck(B) (ms/item)	open(B) (ms/item)		
W	PFE	BTX	BEAT++	PFE	BTX	BEAT++	BTX	PFE	BTX	BEAT++
765	0.035	0.035	0.039	0.963	0.644	0.642	0.097	0.721	0.171	0.171
1,580	0.029	0.029	0.039	1.120	0.722	0.722	0.097	0.721	0.171	0.172
3,063	0.025	0.025	0.039	1.278	0.801	0.800	0.098	0.721	0.171	0.171
256	0.022	0.022	0.039	1.436	0.880	0.880	0.098	0.722	0.171	0.171

Table 5: Mean running times of the batch-size-dependent phases, together with the isolated ciphertext-proof verification cost $\text{ctxtCheck}(B)$.